



LOGICA DE PROGRAMACION

AUTÓNOMO 2

Desarrollo de Software / Configuración del entorno

Autores

Matthew Adrián Narváez Paredes

Profesora

Lilian Marlene Aman Ramos

Fecha de entrega del Proyecto

5 de octubre de 2025

Aprendizaje Autónomo 2

Paso 1: Inicio del desarrollo del software seleccionado

Descripción general del proyecto

El proyecto seleccionado para este Aprendizaje Autónomo 2 consiste en el desarrollo de un programa interactivo titulado “Adivina el Número”, implementado en el lenguaje de programación Python dentro del entorno Visual Studio Code.

El objetivo principal del programa es permitir que el usuario participe en un juego sencillo de adivinanza, donde el sistema genera de forma aleatoria un número entre 1 y 100, y el jugador debe intentar descubrirlo mediante sucesivos intentos.

Cada vez que el usuario ingresa un número, el sistema analiza la respuesta e informa si el número introducido es muy alto, muy bajo o si ha acertado.

Este programa fue elegido por su facilidad para integrar los principales temas vistos en la unidad: estructuras lógicas, estructuras repetitivas, manejo de excepciones y control de flujo del programa, además de representar una aplicación práctica y funcional de los fundamentos de la programación estructurada.

Configuración del entorno de desarrollo

Para llevar a cabo el desarrollo del programa, se configuró el entorno de trabajo de la siguiente manera:

- **Lenguaje de programación:** Python
- **Herramienta utilizada:** Visual Studio Code
- **Librerías empleadas:** random (para generar el número aleatorio)
- **Sistema operativo:** Windows 11
- **Extensiones adicionales:** Python Extension Pack para Visual Studio Code

El entorno fue verificado mediante la ejecución de scripts de prueba básicos, asegurando el correcto funcionamiento del intérprete y la consola de salida.

Codificación inicial

El código inicial del programa fue desarrollado con una estructura ordenada y comentada, garantizando una comprensión clara del funcionamiento de cada bloque.

A continuación, se describen las principales funcionalidades implementadas en esta primera fase:

1. **Generación del número aleatorio:**

Se utilizó la función `random.randint(1, 100)` para crear el número que el usuario debe adivinar.

2. **Captura de datos del usuario:**

Mediante la función `input()`, el programa solicita un número al jugador.

3. **Conversión y validación:**

El valor ingresado se convierte a entero con `int()`. Si el usuario ingresa un dato no válido, se captura el error con `try-except`, evitando que el programa se detenga.

4. **Comparación de valores:**

Se utiliza una estructura condicional `if-elif-else` para determinar si el número ingresado es menor, mayor o igual al número secreto.

5. **Repetición del proceso:**

Con un bucle `while`, el programa continúa solicitando números hasta que el jugador adivine correctamente.

6. **Contador de intentos:**

Se emplea una variable `intentos` que aumenta con cada número ingresado, permitiendo informar al usuario cuántos intentos necesitó para ganar.

7. **Mensajes de salida:**

Se añadieron mensajes descriptivos y emojis para mejorar la interacción con el usuario y la presentación visual en consola.

Diagramas de flujo

Como parte de esta primera fase se elaboraron dos diagramas de flujo que representan la lógica del programa:

- **Diagrama general:** muestra el proceso completo desde la generación del número hasta el mensaje de acierto.
- **Diagrama de validación:** refleja el manejo del error cuando el usuario ingresa un valor no numérico.

Estos diagramas permiten visualizar el flujo lógico del programa, las decisiones que se ejecutan en cada intento y el manejo de las excepciones dentro del proceso.

- **Herramienta utilizada:** Raptor

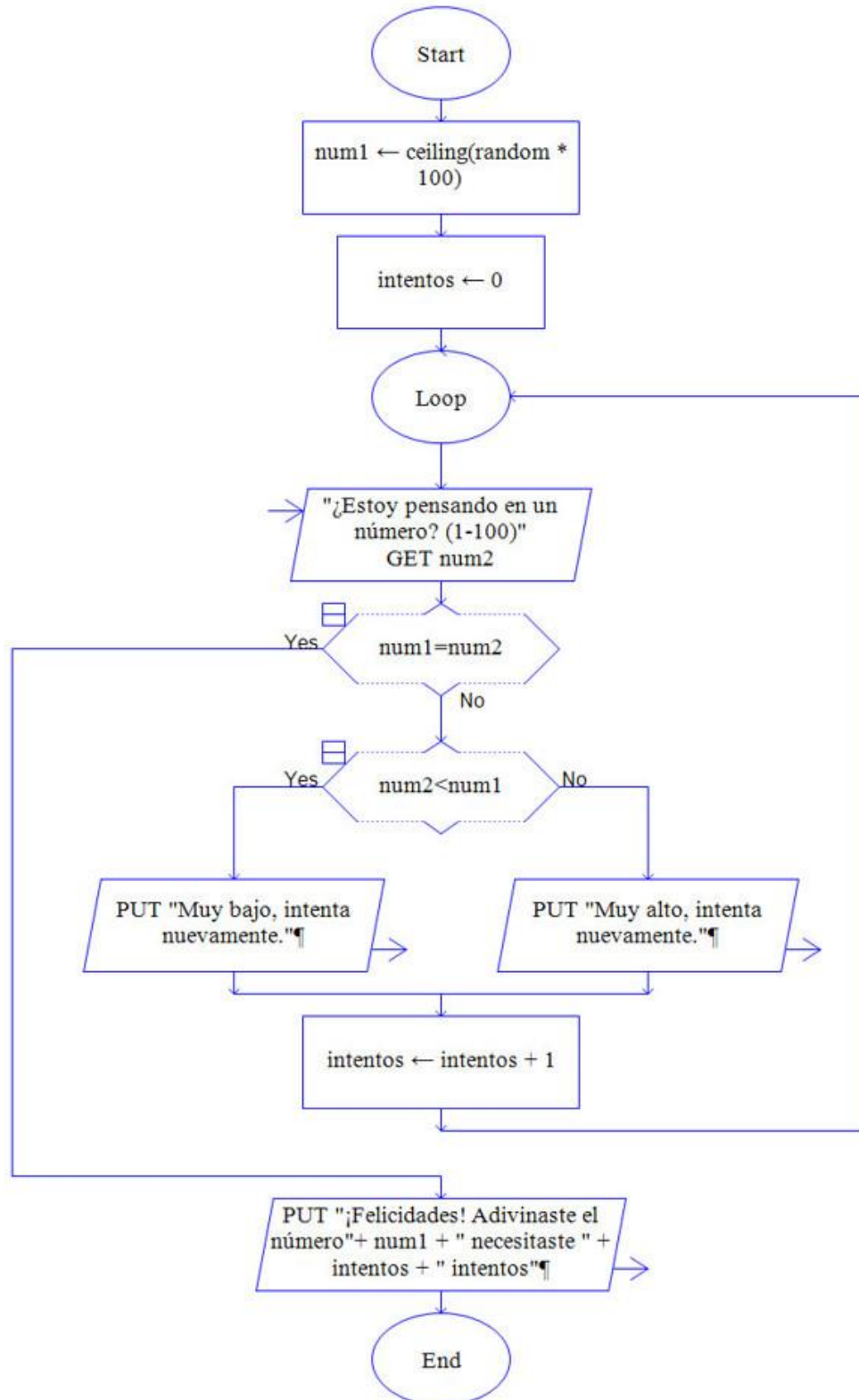


Figura 1. Diagrama de Flujo

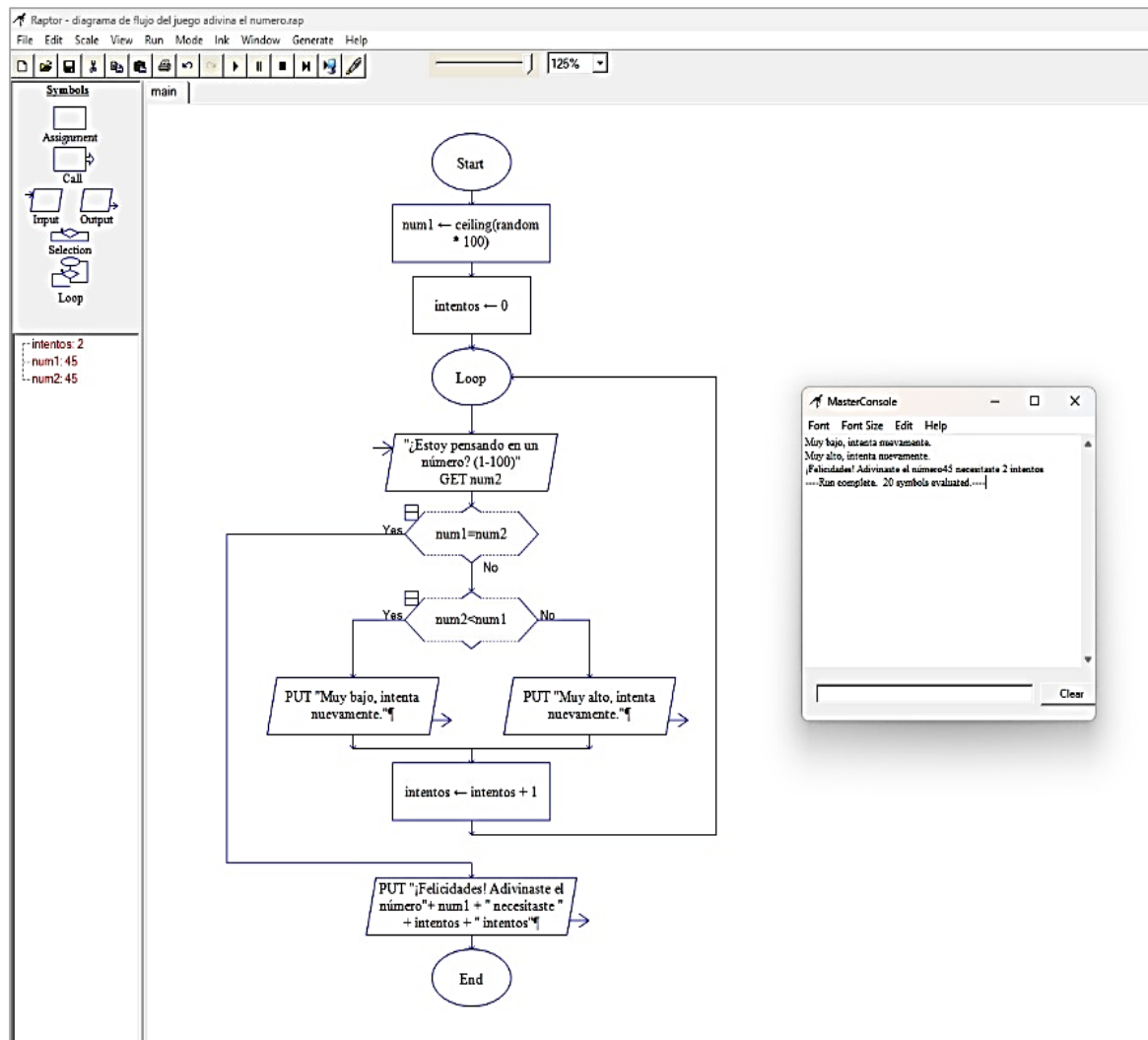


Figura 2. Diagrama de Flujo con sus Probabilidades

Control de versiones y repositorio GitHub

Con el propósito de aplicar buenas prácticas de desarrollo, se creó un repositorio en GitHub, donde se registran los avances del proyecto.

En este repositorio se almacenan:

- El código fuente del programa: main.py
- Los diagramas de flujo: diagrama_general.png y diagrama_validacion.png
- Los videos explicativos y demostrativos del funcionamiento

El uso de GitHub permitió mantener un respaldo del código, documentar cada versión y facilitar la presentación final del trabajo.

<https://github.com/manarvaez207/mi-juego-python>

Diagrama de Arquitectura de Capas: Adivina el Número

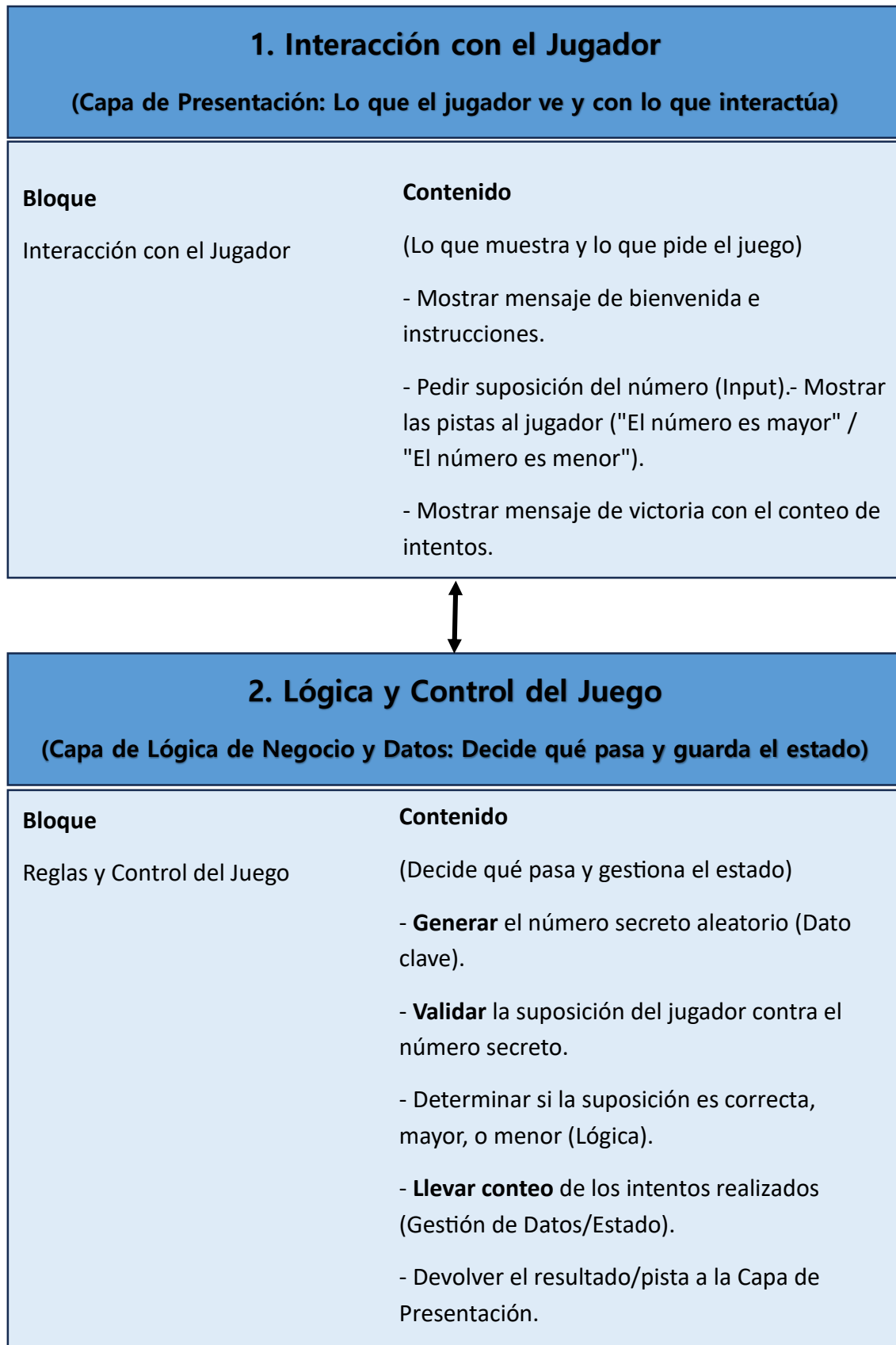


Figura 3. Diagrama de Arquitectura - Capas

Conclusión del Paso 1

En esta etapa se logró preparar todo el entorno necesario para el desarrollo del programa, diseñar la lógica inicial, generar los diagramas de flujo y dejar listo el repositorio donde se documentarán los siguientes avances.

El proyecto “Adivina el Número” cumple con los requerimientos planteados para el inicio del desarrollo, demostrando el uso adecuado de estructuras lógicas, repetitivas y manejo de errores, bases fundamentales de la programación estructurada en Python.

Referencias (APA 7ª edición)

Python Software Foundation. (2024). *Python (versión 3.11)* [Software]. Python.org. <https://www.python.org>

Python Software Foundation. (s. f.). *Módulo random — Generación de números aleatorios* [Documentación]. En *Python 3.11 Documentation*. <https://docs.python.org/3/library/random.html>

Microsoft. (2024). *Visual Studio Code: Code editing. Redefined* [Software]. Microsoft Corporation. <https://code.visualstudio.com>

GitHub, Inc. (2024). *GitHub: Where the world builds software* [Plataforma web]. <https://github.com>

Venigalla, A. S. M., & Chimalakonda, S. (2021). *What's in a GitHub repository? — A software documentation perspective*. arXiv. <https://arxiv.org/abs/2102.12727>

Tan, W. S., Wagner, M., & Treude, C. (2022). *Detecting outdated code element references in software repository documentation*. arXiv. <https://arxiv.org/abs/2212.01479>

Python Software Foundation. (s. f.). *pydoc — Módulo de documentación del lenguaje Python* [Documentación]. En *Python 3.11 Documentation*. <https://docs.python.org/3/library/pydoc.html>

Aprendizaje Autónomo 2

Paso 2: Desarrollo del programa seleccionado

Introducción

En el Paso 2, se desarrolló la segunda fase del proyecto del curso, implementando las funcionalidades en el programa “Adivina el Número”.

Durante esta etapa se integraron estructuras lógicas (if, elif, else), estructuras repetitivas (while), y manejo de excepciones (try/except), junto con comentarios explicativos en las partes más complejas del código.

Además, se prepararon las evidencias de entrega: videos explicativos y demostrativos, y la carga del código al repositorio de GitHub.

Objetivos del Paso 2

- Implementar funcionalidades que evidencien el dominio de estructuras lógicas y repetitivas.
- Incorporar manejo de excepciones y validaciones de entrada para mayor robustez.
- Documentar el código con comentarios claros en las secciones más relevantes.
- Generar las evidencias requeridas: video explicativo, video de demostración y carga en GitHub.

1. Funcionalidades Implementadas

1.1 Lógica principal del juego

- Generación de un número aleatorio mediante `random.randint(1, 100)`.
- Comparación con la entrada del usuario usando `if / elif / else`.
- Mensajes dinámicos: “Muy bajo”, “Muy alto” y “¡Felicidades!”, junto con un contador de intentos.

1.2 Estructura repetitiva

- Uso de un ciclo `while` not adivinado: que mantiene el juego activo hasta acertar.
- Incremento del contador intentos en cada iteración válida.

1.3 Manejo de excepciones y validación

- Implementación de un bloque try / except ValueError para evitar fallos con entradas no numéricas.
- Validación del rango del número (1–100) para reforzar la confiabilidad del programa.

1.4 Comentarios y documentación

- Comentarios descriptivos al inicio del archivo (autor, propósito, fecha).
- Comentarios antes de los bloques más relevantes: generación del número, validación de entrada, manejo de errores y control de flujo.

1.5 Mejoras opcionales

- Implementación sugerida: niveles de dificultad, límite de intentos, registro de partidas o reinicio del juego sin cerrar el programa.

```
12 # Generamos el número aleatorio que el jugador debe adivinar
13 numero_secreto = random.randint(1, 100)
14
15 # Inicializamos variables
16 intentos = 0
17 adivinado = False
18
19 # Bucle principal del juego: se repite hasta que el usuario adivine
20 while not adivinado:
21     try:
22         # Solicitamos al usuario que ingrese un número
23         entrada = input(" Ingresa tu número: ")
24         numero = int(entrada) # Convertimos la entrada en entero
25         intentos += 1 # Sumamos un intento
26
27         # Verificamos si el número es menor, mayor o igual al secreto
28         if numero < numero_secreto:
29             print(" ▼ Muy bajo, intenta nuevamente.\n")
30         elif numero > numero_secreto:
31             print(" ▲ Muy alto, intenta nuevamente.\n")
32         else:
33             print(f" ¡Felicidades! Adivinaste el número {numero_secreto} en {intentos} intentos.")
34             adivinado = True # Salimos del bucle
35
36     except ValueError:
37         # Capturamos errores si el usuario ingresa algo que no sea número
38         print(" Error: Debes ingresar un número entero válido.\n")
39
40 # Mensaje final
41 print(" Gracias por jugar. ¡Hasta la próxima!")
```

Figura 1. Código Principal - Python

2. Fragmento de Código Principal



```
Help      ← →      🔍 mi-juego-python

Welcome  adivina el numero.py U X

adivina el numero.py > ...
1  # Juego: Adivina el Número
2  # Autor: Matthew Narvaez
3  # Desarrollado para el Aprendizaje Autónomo 2
4  # Lenguaje: Python
5  # Herramienta: Visual Studio Code
6
7  import random # Importamos la librería para generar números aleatorios
8
9  print("Bienvenido al juego: Adivina el Número")
10 print("Estoy pensando en un número entre 1 y 100...")
11
12 # Generamos el número aleatorio que el jugador debe adivinar
13 numero_secreto = random.randint(1, 100)
14
15 # Inicializamos variables
16 intentos = 0
17 adivinado = False
18
19 # Bucle principal del juego: se repite hasta que el usuario adivine
20 while not adivinado:
21     try:
22         # Solicitamos al usuario que ingrese un número
23         entrada = input(" Ingresa tu número: ")
24         numero = int(entrada) # Convertimos la entrada en entero
25         intentos += 1 # Sumamos un intento
26
27         # Verificamos si el número es menor, mayor o igual al secreto
28         if numero < numero_secreto:
29             print(" ▼ Muy bajo, intenta nuevamente.\n")
30         elif numero > numero_secreto:
31             print(" ▲ Muy alto, intenta nuevamente.\n")
32         else:
33             print(f" ¡Felicidades! Adivinaste el número {numero_secreto} en {intentos} intentos.")
34             adivinado = True # Salimos del bucle
35
36     except ValueError:
37         # Capturamos errores si el usuario ingresa algo que no sea número
38         print(" Error: Debes ingresar un número entero válido.\n")
39
40 # Mensaje final
41 print(" Gracias por jugar. ¡Hasta la próxima!")

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
```

Figura 2. Fragmento Código Principal - Python

Comentarios destacados:

- El try/except evita cierres inesperados del programa.
- La validación de rango mejora la experiencia del usuario.
- Cada bloque incluye comentarios para facilitar la comprensión y evaluación.

```
21     try:
22         # Solicitamos al usuario que ingrese un número
23         entrada = input(" Ingresa tu número: ")
24         numero = int(entrada) # Convertimos la entrada en entero
25         intentos += 1 # Sumamos un intento
26
36     except ValueError:
37         # Capturamos errores si el usuario ingresa algo que no sea número
38         print(" Error: Debes ingresar un número entero válido.\n")
39
40 # Mensaje final
41 print(" Gracias por jugar. ¡Hasta la próxima!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Figura 3. Fragmento Código Principal - Python

3. Documentación del Código

El archivo adivina_el_numero.py contiene:

- **Cabecera:** información del autor, fecha y propósito.
- **Comentarios por bloque:** antes de cada sección lógica.
- **Explicación adicional:** en el manejo de excepciones y validaciones complejas.

Ejemplo de comentario detallado:

VALIDACIÓN: Si el usuario ingresa texto o símbolos, int() lanzará ValueError.

Capturamos ese error para mostrar un mensaje al usuario sin detener el programa.

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Ingresa tu número: 20
▲ Muy alto, intenta nuevamente.

Ingresa tu número: 15
▼ Muy bajo, intenta nuevamente.

Ingresa tu número: 18
▼ Muy bajo, intenta nuevamente.
```

Figura 4. Fragmento Código Principal - Python

4. Pruebas Realizadas

```
Ingresa tu número: 18
▼ Muy bajo, intenta nuevamente.

Ingresa tu número: 19
¡Felicidades! Adivinaste el número 19 en 4 intentos.
Gracias por jugar. ¡Hasta la próxima!
PS C:\Users\Milton\OneDrive\Escritorio\mi-juego-python>
```

Figura 6. Fragmento Código Principal - Python

5. Videos Requeridos

Video explicativo del avance del Juego:

- Presentación del proyecto y objetivos.
- Recorrido del código en VS Code (lógica, bucle, excepciones, validaciones).
- Breve mención de diagramas de flujo.
- Estructura del repositorio en GitHub.
- Conclusión y aprendizajes.

Video demostrativo:

- Demostración del Diagrama de Flujo del juego "Adivina el Número".
- Demostración del Diagrama de Arquitectura de Capas "Adivina el Número".
- Visualización opcional del repositorio GitHub.

6. Organización del Repositorio GitHub

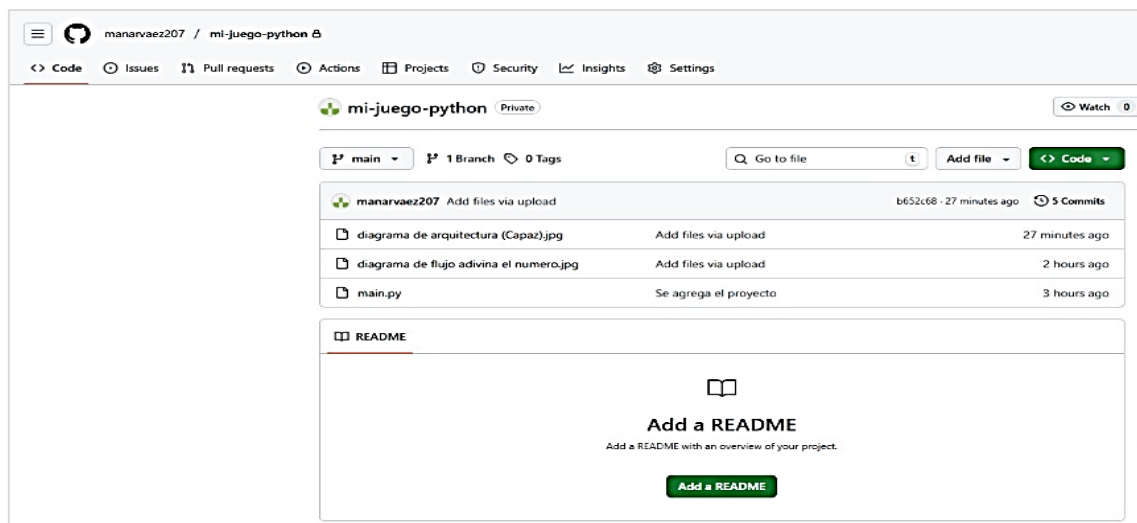


Figura 7. Repositorio GitHub

Conclusión del Paso 2

El Paso 2 se completó satisfactoriamente, demostrando la comprensión y aplicación de los contenidos de la tercera unidad: estructuras lógicas, repetitivas, manejo de excepciones y documentación del código.

Con las evidencias solicitadas (código, pruebas, videos y repositorio), el proyecto cumple con todos los criterios de evaluación establecidos.

Referencias (APA 7.ª edición)

Microsoft. (2025). *Visual Studio Code Documentation*. Microsoft Docs.
<https://code.visualstudio.com/docs>

Python Software Foundation. (2025). *Python 3.12 Documentation*.
<https://docs.python.org/3/>

GitHub, Inc. (2025). *GitHub Docs: Understanding the GitHub flow*.
<https://docs.github.com/en/get-started/quickstart/github-flow>

W3Schools. (2025). *Python Tutorial*.
<https://www.w3schools.com/python/>

Real Python. (2025). *Python Exceptions: An Introduction*.
<https://realpython.com/python-exceptions/>